Native fork join: work stealing by context switching

Mike Rainey
Carnegie Mellon University
Pittsburgh, PA, USA
me@mike-rainey.site

Abstract

TaskPaRTS is a C++ library for fork-join parallelism that requires no compiler support. At its core is a technique we call *native fork join*, which delivers work-stealing scheduling by context switching. A fork saves the continuation of the parent task as a handful of machine registers that point into the parent's own stack. When the join finds that the second branch was stolen, the worker neither blocks nor runs other work on top of the parent's frame: it leaves the parent's stack suspended and returns to its scheduler. Whichever worker finishes the last branch then resumes the parent, possibly on a different thread. When nothing is stolen, both branches run on the parent's stack, like ordinary calls, and the fork allocates no memory. In this short paper, we present the algorithm in pseudocode, explain why it is safe, and describe what it means for programs that use it. A companion implementation, micro taskparts, implements the same algorithm in a few hundred lines of C++.

1 Challenge: the join, in a library

We are to implement binary fork join, which runs two thunks, possibly in parallel, and returns once both have finished.

```
fork2join(f1, f2 : () → void) → void
```

For example:

```
fib(n : int) → int {
  if (n < 2) return n
  fork2join(λ () ⇒ {a = fib(n-1)},
            λ () ⇒ {b = fib(n-2)})
  return a + b }
```

Work stealing [5, 8] is well suited to scheduling such programs. Our constraint is that we are writing a library: there is no compiler support, and client code is ordinary C++, with lambdas for thunks.

A library work stealer typically implements `fork2join` by *child stealing*: the second branch becomes a task that thieves may take.

```
fork2join(f1, f2 : () → void) → void {
  t = new_task(f2); push(t)   // t is now stealable
  f1()
  if (pop() == t) { f2(); return }   // not stolen
  while (!t.done)                    // stolen: wait for thief
    { u = steal(); if (u != null) u.run() } }
```

The common case, where `t` is not stolen, is cheap. The difficulty is the join when `t` was stolen. The parent's frame sits on the worker's thread stack, and must stay there until the join completes. So the worker can either block, and sit idle, or, as TBB and ParlayLib do [4, 9], *help*: run other tasks on top of the blocked frame, as in the loop above. Helping keeps the worker busy, but the parent can resume only once the worker has unwound back down to its frame, even if the thief finished long before. Cilk [7, 13] avoids waiting by stealing the parent's continuation instead of the child. But then the thief must resume a frame that sits on the victim's stack, which in Cilk relies on compiler support and a cactus stack [10].

Native fork join, which is at the core of TaskPaRTS [12], takes a different route. When a worker finds at a join that the second branch was stolen, it *suspends the parent's stack* and returns to its scheduler, which looks for other work. The parent's continuation is a saved register context that points into the suspended stack. Whichever worker finishes the last branch restores this context, and so the parent continues on that worker's thread. No worker waits at a join, and no two workers ever run on the same stack at once.

We present the algorithm in Section 2, trace its two cases in Section 3, explain why it is safe in Section 4, and discuss its costs and its consequences for client code in Section 5. The appendices give machine-level details, the work-stealing deque, and a map from the pseudocode to TaskPaRTS and to micro taskparts, a minimal C++ implementation that accompanies this paper.

2 Native fork join

Contexts. Our one low-level primitive is the *context*. A context holds what the C calling convention requires a function call to preserve: the callee-saved registers, the stack pointer, and a return address. That is 8 words on x86-64 and 21 on AArch64 (Appendix A). `save(`k`)` stores its caller's context in k and returns **`false`**; `restore(`k`)` resumes that call to `save`, which this time returns **`true`**. The pair behaves like `setjmp` and `longjmp`, except that the thread that restores a context need not be the one that saved it, and the context may point into any stack. To the compiler, `save` is an ordinary call (one that returns twice, like `setjmp`), so it spills any live caller-saved registers before the call. As a result, a save or a restore is a handful of loads and stores, with no system call. We write `swap(`k_1`, `k_2`)` for **`if`** `(!save(`k_1`)) restore(`k_2`)`, which saves the current context in k_1 and switches to k_2.

Vertices and workers. Following the DAG calculus [1], the scheduler represents a computation as a DAG whose

```
type action = Init | Resume | Finish
type vertex = {in : atomic int, out : vertex*,
  act : action, k : context, body : () → void}
type worker = {k : context, cur : vertex*, ...}

loop() → void {   // on the worker's own stack
  while (!done) {
    v = pop()
    if (v == null) {
      v = steal()
      if (v != null) schedule(v)
      continue }
    me.cur = v
    if (v.act == Init) {
      v.k = new_context(new_stack(), enter)
      v.act = Finish }
    swap(me.k, v.k)   // run v until it yields
    if (v.act == Finish) notify(v) } }

enter() → void {   // first frame on a fresh stack
  me.cur.body(); restore(me.k) }

notify(v : vertex*) → void {
  u = v.out   // read before u.in can reach 0
  if (u == null) { done = true; return }   // root
  if (fetch_and_add(u.in, -1) == 1) schedule(u) }
```

Figure 1. The scheduler

vertices each have an *in-counter*, which counts unfinished predecessors, and an *out-set*, which, in fork join, is a single successor (Figure 1). A vertex also has a context, `k`, and an action, `act`, which tells the scheduler what to do with the vertex. `Init` means that the vertex has never run; `Resume`, that it is suspended inside `fork2join`, with its context saved in `k`; and `Finish`, that it will have finished the next time it hands control back to the scheduler.

Each worker runs a scheduler loop on its thread's own stack. While the worker runs a vertex, `cur`, it keeps the loop's context in `k`. Client code runs on separate *task stacks*. We write `me` for the worker that runs the caller, which we look up again at each use, because the worker can change across a call to `fork2join` (Section 5). Each worker also holds its ready vertices: `schedule(`v`)` makes v ready, `pop()` takes back the vertex scheduled most recently, if any, and `steal()` takes the oldest ready vertex of another worker, chosen at random. The vertex scheduled most recently can never be stolen (Appendix B), and `fork2join` relies on this.

The scheduler. The scheduler loop (Figure 1) is the only place where vertices start, resume, and finish. It pops a vertex, or else steals one and makes it ready, and then makes the vertex current. If the vertex has never run, the loop gives it

```
fork2join(f1, f2 : () → void) → void {
  v0 = me.cur
  v1 = vertex{out = v0, body = λ () ⇒ {}}
  v2 = vertex{out = v0, body = f2}
  if (save(v0.k))                                  // (1)
    { v0.act = Finish; return }
  v0.act = Resume; v0.in = 2                       // (2)
  schedule(v2); schedule(v1)
  v1.act = Resume                                  // (3)
  swap(v1.k, me.k); v1.act = Finish
  f1()
  if (pop() == null) restore(me.k)                 // (4)
  v2.act = Resume; schedule(v2)                    // (5)
  swap(v2.k, me.k); v2.act = Finish
  f2()
  swap(v0.k, me.k); v0.act = Finish }              // (6)
```

Figure 2. Fork join

a fresh stack and a context that, once restored, calls `enter` at the top of that stack. `enter` runs the body of the vertex, and then switches to the scheduler of whichever worker it is on by then. The loop then switches to the vertex, and regains control when the vertex yields. If the action of the vertex is then `Finish`, the loop notifies its successor: it decrements the successor's in-counter, and the worker whose decrement reaches zero schedules the successor. A call to `fork2join` from outside the scheduler starts the workers with a root vertex that makes the call, and the computation ends when the root, which has no successor, finishes.

Fork join. In `fork2join` (Figure 2), the parent, v_0, is the current vertex. The function creates two vertices, v_1 and v_2, both with v_0 as their successor. v_2 runs f_2, and v_1 has an empty body. v_1 stands for the parent while the parent runs f_1 inline, so that any `fork2join` nested inside f_1 uses the context and in-counter of v_1, and leaves those of v_0 alone. The steps are as follows.

(1) Save the parent's context in v_0`.k`. If v_2 is stolen, this is where the parent resumes.
(2) Set the in-counter of v_0 to 2, and schedule v_2, then v_1, which leaves v_2 stealable and v_1 not.
(3) Yield to the scheduler, saving the context in v_1`.k`. The scheduler pops v_1 and resumes it at once, so execution continues after the `swap`, with v_1 as the current vertex. Run f_1 inline.
(4) Pop. If the pop fails, v_2 was stolen. Then abandon the current stack by switching to the scheduler without saving anything. The stack stays suspended at step 1, and the scheduler notifies v_0 that v_1 has finished.
(5) Otherwise, the pop returned v_2. Schedule v_2 again, so that it is the vertex scheduled most recently and cannot

```
on task stack S                 in the scheduler of worker A
(1) save(v₀.k)
(2) schedule(v₂)
    schedule(v₁)
(3) swap(v₁.k, A.k)   → pop v₁, resume it
    f₁()
(4) pop() returns v₂
(5) schedule(v₂)
    swap(v₂.k, A.k)   → notify: v₀.in = 1
                        pop v₂, resume it
    f₂()
(6) swap(v₀.k, A.k)   → notify: v₀.in = 0
                        schedule, pop, resume v₀
    return
```

```
worker A                        worker B
(1)-(3); f₁() on S              steal v₂ from A
                                fresh stack T; f₂() on T
(4) pop() returns null
    restore(A.k)
    notify: v₀.in = 1
    find other work ...
                                f₂() returns
                                restore(B.k)
                                notify: v₀.in = 0
                                schedule, pop v₀
                                restore(v₀.k): the save
                                at (1) returns, on S
```

Figure 3. The two cases: v_2 not stolen (top), and v_2 stolen by worker B, which finishes last (bottom)

be stolen, and yield. The scheduler notifies v_0 that v_1 has finished, then pops v_2 and resumes it here. Run f_2 inline.

(6) Yield once more, saving the context in v_0.k. The scheduler notifies v_0 that v_2 has finished, which makes v_0 ready, then pops v_0 and resumes it here, and `fork2join` returns.

3 The two cases

Figure 3 traces both cases of `fork2join`.

Not stolen. One worker, A, does everything. f_1 and f_2 both run on the parent's task stack, S, as ordinary calls, and the only overhead is three round trips through the scheduler.

Stolen. Worker B steals v_2 from A while A runs f_1, gives v_2 a fresh stack, T, and runs f_2 there. When the pop at step 4 fails, A abandons S, and its scheduler notifies v_0 that v_1 has finished, which brings the in-counter of v_0 to 1. A then looks for other work. When f_2 returns, B's scheduler notifies v_0 that v_2 has finished, which brings the in-counter to 0. So B schedules v_0, pops it, and restores v_0.k. The call to `save` at step 1 returns a second time, on stack S as before, but now on worker B, and `fork2join` returns there. Had A finished last, A would have resumed the parent itself.

So a call to `fork2join` can begin on one thread and return on another. The stack stays the same throughout; only the thread that runs it changes.

4 Why it is safe

The correctness of the algorithm rests on four facts.

Only vertices that have never run are stolen. `fork2join` schedules v_1 (step 2) and v_2 (step 5) before it saves their contexts. At that moment, each is the vertex scheduled most recently, so it cannot be stolen, and its own worker pops it only after the `swap` to the scheduler has saved its context. The only vertices that thieves ever see are those like v_2 at step 2, whose action is `Init`.

Completion is reported from the scheduler. As soon as the in-counter of v_0 reaches zero, another worker may resume v_0 on stack S. So the decrement on behalf of v_1 must happen after A has left S, which is why step 4 switches to the scheduler instead of decrementing in place. Similarly, v_1 and v_2 live in the frame of v_0 on S, so `notify` reads the successor of a vertex before decrementing, and does not touch the vertex afterward.

The pop at step 4 is correct even if f_1 moved. f_1 returns on a different worker only if a vertex forked inside f_1 was stolen. Thieves take the oldest ready vertex, and v_2 is older than any vertex forked by f_1. So by then v_2 has been stolen as well, and the pop correctly fails.

A fresh stack is freed only after its last use. A stolen v_2 owns its fresh stack, and v_2 lives in the frame of v_0, so the stack is freed when the parent's call to `fork2join` returns. By then, f_2 has finished, and its worker has switched back to its scheduler.

5 Costs and consequences

Costs. A fork whose second branch is not stolen makes three round trips through the scheduler (steps 3, 5, and 6), each a save and a restore, and allocates nothing, because v_1 and v_2 live in the frame of `fork2join`. In this case, no other worker can reach the in-counter of v_0, so TaskPaRTS updates it with plain loads and stores, rather than with atomic operations. The round trips are not essential to the technique. They keep the scheduler loop the only place where vertices start, resume, and finish, and they could be optimized away. A steal allocates one fresh stack, 512 KB in TaskPaRTS, which is touched only as deep as f_2 recurses, and is freed when the parent returns from `fork2join`.

Consequences for client code. Code that follows a call to `fork2join` may run on a different thread from the code that precedes it. So client code must not hold a lock across `fork2join`, nor expect a thread-local variable or the thread id to be the same afterward. More subtly, compilers assume that

a function never changes threads, and so may reuse a thread-local address computed before the call. Then even a fresh read of a thread-local variable after `fork2join`, in the same function, can return the old thread's copy. Reading thread-local state through a function that the compiler can neither inline nor treat as pure avoids this problem. Other consequences follow from the use of task stacks. Task stacks have a fixed size, and no guard page by default. Exceptions must not escape $\texttt{f}_1$ or $\texttt{f}_2$, because unwinding cannot cross from a task stack into the scheduler. Backtraces end at the bottom of a task stack. And sanitizers, such as AddressSanitizer and ThreadSanitizer, are not told about the stack switches, so they may report false errors.

Implementations. TaskPaRTS implements the algorithm of Figures 1 and 2, and adds lock-free deques [3, 6], elastic scheduling, instrumentation, and CPU pinning around it. Micro taskparts implements the same algorithm and nothing else, in about 440 lines of C++, including the assembly for x86-64 and AArch64. Appendix C maps the pseudocode to both.

References

[1] U. A. Acar, A. Charguéraud, M. Rainey, and F. Sieczkowski. Dag-calculus: A calculus for parallel computation. In *The 26th ACM SIGPLAN International Conference on Functional Programming*, ICFP '16. ACM, 2016.

[2] Arm Limited. Procedure call standard for the Arm 64-bit architecture (AArch64). https://github.com/ARM-software/abi-aa.

[3] N. S. Arora, R. D. Blumofe, and C. G. Plaxton. Thread scheduling for multiprogrammed multiprocessors. In *SPAA '98*, pages 119–129, 1998.

[4] G. E. Blelloch, D. Anderson, and L. Dhulipala. ParlayLib – a toolkit for parallel algorithms on shared-memory multicore machines. In *SPAA '20*, pages 507–509, 2020.

[5] R. D. Blumofe and C. E. Leiserson. Scheduling multithreaded computations by work stealing. *J. ACM*, 46:720–748, Sept. 1999.

[6] D. Chase and Y. Lev. Dynamic circular work-stealing deque. In *SPAA '05*, pages 21–28, 2005.

[7] M. Frigo, C. E. Leiserson, and K. H. Randall. The implementation of the Cilk-5 multithreaded language. In *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation*, PLDI '98, pages 212–223. ACM, 1998.

[8] R. H. Halstead, Jr. Implementation of Multilisp: Lisp on a Multiprocessor. In *Proceedings of the 1984 ACM Symposium on LISP and functional programming*, LFP '84, pages 9–17. ACM, 1984.

[9] Intel. Intel threading building blocks, 2011. https://www.threadingbuildingblocks.org/.

[10] I.-T. A. Lee, S. Boyd-Wickizer, Z. Huang, and C. E. Leiserson. Using memory mapping to support cactus stacks in work-stealing runtime systems. In *Proceedings of the 19th International Conference on Parallel Architectures and Compilation Techniques*, PACT '10, pages 411–420, New York, NY, USA, 2010. ACM.

[11] H. J. Lu, M. Matz, M. Girkar, J. Hubička, A. Jaeger, and M. Mitchell. System V application binary interface: AMD64 architecture processor supplement. https://gitlab.com/x86-psABIs/x86-64-ABI.

[12] M. Rainey. TaskPaRTS: a task-parallel run-time system for C++. https://github.com/mikerainey/taskparts.

[13] T. B. Schardl, W. S. Moses, and C. E. Leiserson. Tapir: Embedding fork-join parallelism into llvm's intermediate representation. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 249–265, 2017.

```
save:                       restore:
  movq %rbx, 0(%rdi)          movq 0(%rdi), %rbx
  movq %rbp, 8(%rdi)          movq 8(%rdi), %rbp
  movq %r12, 16(%rdi)         movq 16(%rdi), %r12
  movq %r13, 24(%rdi)         movq 24(%rdi), %r13
  movq %r14, 32(%rdi)         movq 32(%rdi), %r14
  movq %r15, 40(%rdi)         movq 40(%rdi), %r15
  leaq 8(%rsp), %rdx          movq 48(%rdi), %rsp
  movq %rdx, 48(%rdi)         movl $1, %eax
  movq (%rsp), %rdx           jmpq *56(%rdi)
  movq %rdx, 56(%rdi)
  xorl %eax, %eax
  ret
```

Figure 4. Saving and restoring a context on x86-64

```
type worker = {k : context, cur : vertex*,
  next : vertex*, dq : deque of vertex*}

schedule(v : vertex*) → void {
  if (me.next != null) push_bottom(me.dq, me.next)
  me.next = v }

pop() → vertex* {
  v = me.next
  if (v != null) { me.next = null; return v }
  return pop_bottom(me.dq) }

steal() → vertex* {
  return pop_top(random_other_worker().dq) }
```

Figure 5. Ready vertices: a private slot over a deque

A Machine contexts

A context holds the registers that the calling convention requires a callee to preserve [2, 11], plus the stack pointer and the address at which to resume:

	x86-64	AArch64
callee-saved	`rbx`, `rbp`, `r12`–`r15`	`x19`–`x29`, `d8`–`d15`
stack pointer	`rsp`	`sp`
resume address	return address	`lr` (`x30`)
total	8 words	21 words

Caller-saved registers need no saving: to the compiler, `save` is an ordinary call, so the compiler has already spilled any live caller-saved registers before calling it. Figure 4 shows both functions for x86-64. `save` records, as the stack pointer to restore, the value the stack pointer will have once `save` has returned, and, as the resume address, its own return address. `restore` reloads the registers, switches stacks, and jumps to the resume address with 1 in the return register. The AArch64 versions are analogous, and resume with `ret` to the restored `lr`.

Declaring save to the compiler. Because `save` can return twice, it must be declared as such to the compiler (in GCC and Clang, with the attribute `returns_twice`). Otherwise, the compiler may keep a value that changes between the two returns in a callee-saved register, or reuse its stack slot for another value, and the second return then sees a stale or clobbered value.

Fresh contexts. To start a function f on a fresh stack s, the scheduler makes a context with `new_context(`s`, `f`)`. The stack pointer of this context is the top of s, aligned to 16 bytes, its resume address is f, and its other registers are zero. On x86-64, the stack pointer is a further 8 bytes down, as if a `call` had pushed a return address. Restoring the context then starts f on s, as if f had been called there. TaskPaRTS builds fresh contexts differently, with the same effect: the scheduler loop calls `save`, and then overwrites the saved stack pointer with the top of a fresh stack. Restoring that context returns from that call to `save` on the fresh stack, where the code that follows calls `enter`.

B Ready vertices

Each worker keeps its ready vertices in a private slot, `next`, in front of a work-stealing deque, `dq` (Figure 5). `schedule(`v`)` puts v in the slot, and moves the slot's previous occupant to the bottom of the deque. `pop` takes from the slot first, and then from the bottom of the deque, and thieves take from the top of the deque. So the vertex scheduled most recently is always in the slot, where no thief can reach it, which is the property that Section 4 relies on.

At step 2 of `fork2join`, the slot is empty: the current vertex was itself popped from it, and any nested `fork2join` calls have taken back what they scheduled. So `schedule(`v_2`)` puts v_2 in the slot, and `schedule(`v_1`)` moves v_2 to the deque. The slot is also empty whenever `notify` or a thief schedules a vertex, so a vertex that reaches the deque is always a v_2 displaced at step 2.

Any work-stealing deque will do. Micro taskparts uses a deque protected by a lock, whereas TaskPaRTS uses a lock-free deque, chosen at compile time among the deque of Arora et al. [3], that of Chase and Lev [6], and a deque by Yao, Westrick, Rainey, and Acar, which supports elastic scheduling.

C From pseudocode to code

Table 1 maps the names in the pseudocode to their counterparts in micro taskparts and in TaskPaRTS. Both are in the TaskPaRTS repository [12]: micro taskparts in the directory `doc/native-fork-join/microtaskparts`, and TaskPaRTS in the files `src/taskparts.cpp` and `include/taskparts/taskparts_internal.hpp`. Both implementations number the steps of `fork2join` as in Figure 2.

pseudocode	micro taskparts	TaskPaRTS
`save`, `restore`	`mtp_context_save`, `mtp_context_restore`	`context_save`, `context_restore`
`vertex`	`vertex`	`native_fork_join_vertex`
`action`	`action`	`continuation_action`
`next`, `dq`	`next`, `deque`	`bottom`, `deque`
`loop`	`worker_loop`	`loop`
`new_context`	`new_context`	`new_continuation`
`enter`	`vertex_entry`	`enter`
`notify`	`notify`	`parallel_notify`
`fork2join`	`fork2join`	`fork2join`

Table 1. Names in the pseudocode, in micro taskparts, and in TaskPaRTS

Beyond micro taskparts, TaskPaRTS adds the following.

- Lock-free deques (Appendix B).
- Elastic scheduling: workers that fail to find work suspend on a semaphore, instead of spinning, and are woken when there is surplus work.
- Non-atomic in-counter updates when the second branch is not stolen (`clone_fast`).
- A persistent pool of workers, and a sink vertex that marks the end of each top-level computation, in place of the `done` flag.
- Statistics, logging in a format that the Perfetto trace viewer reads, and CPU pinning with hwloc.